

Sydney housing price prediction

SIT720 Task 8.1D | 18 September 2026

1. Problem and data collection

This project evaluates regression methods for predicting residential sale prices in Australian dollars. Parramatta, Blacktown and Mosman were selected as contrasting purchasing locations. The Parramatta sample consists mainly of apartments, Blacktown includes houses and apartments, and Mosman contains higher-priced properties. These differences allow the analysis to examine how location, dwelling type and accommodation relate to sale prices.

The dataset contains 119 sold properties: 37 in Parramatta, 43 in Blacktown and 39 in Mosman. Sale information was transcribed from Domain and realestate.com.au listings. Each record includes its source URL and access date to support traceability. The reported prices are listing information rather than independently verified settlement records.

Recorded fields	Use
Sale price (AUD)	Prediction target
Suburb, type, bedrooms, bathrooms, parking, sale date	Model inputs
Address, advertised area, source, access date, property ID	Checking and traceability; excluded from prediction

Collection problems included withheld prices, missing parking, inconsistent area definitions and changing listing details. Withheld-price and retirement listings were excluded. Missing parking stays unknown. Advertised area is retained for checking but excluded because it mixes land, floor and whole-building area. All 119 transcriptions were checked against the viewed listing text; this does not independently verify settlement records.

The sample is convenient rather than random. Disclosed prices may be selective, several units share buildings, and the Mosman house search was supplemented separately. Results should not be presented as official suburb statistics or as reliable estimates for all Sydney properties.

Data and implementation

Application: [Open the housing predictor on GitHub Pages](#)

Dataset and source archive: [Open the submission archive](#)

2. Understanding and preparing the data

A fixed split reserves 24 test properties: ten preselected comparison cases and fourteen additional cases sampled by suburb with seed 42. The remaining 95 support exploration and model selection. The test is not purely random, and nearby units can cross the split. Building-grouped and forward-time tests would be stronger.

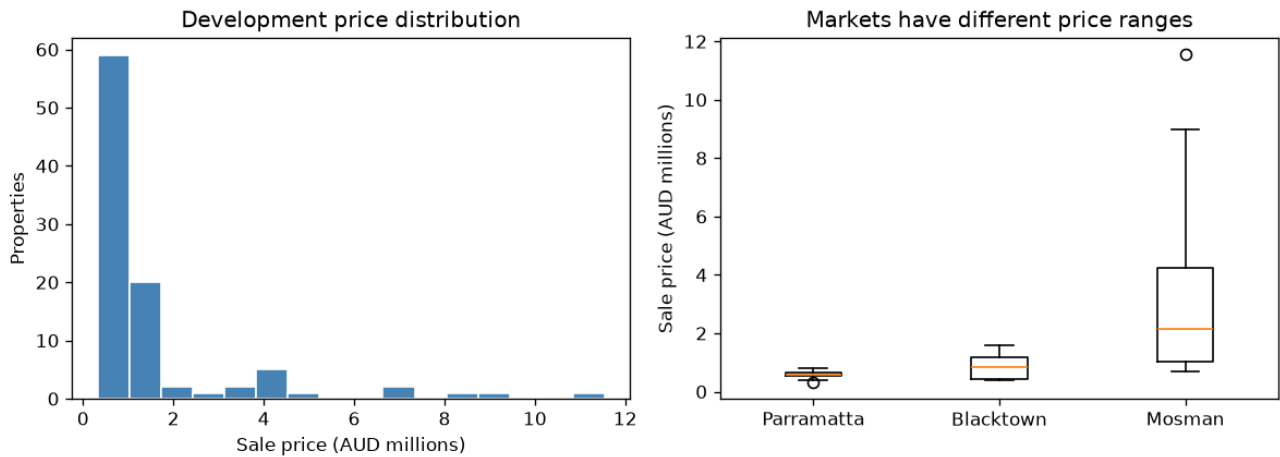


Figure 1. Development data only. The right tail contains expensive Mosman houses.

Development median prices are \$572,000 in Parramatta, \$854,000 in Blacktown and \$2,152,500 in Mosman. Prices are right-skewed. Expensive genuine sales are retained; removing them would conceal important failures.

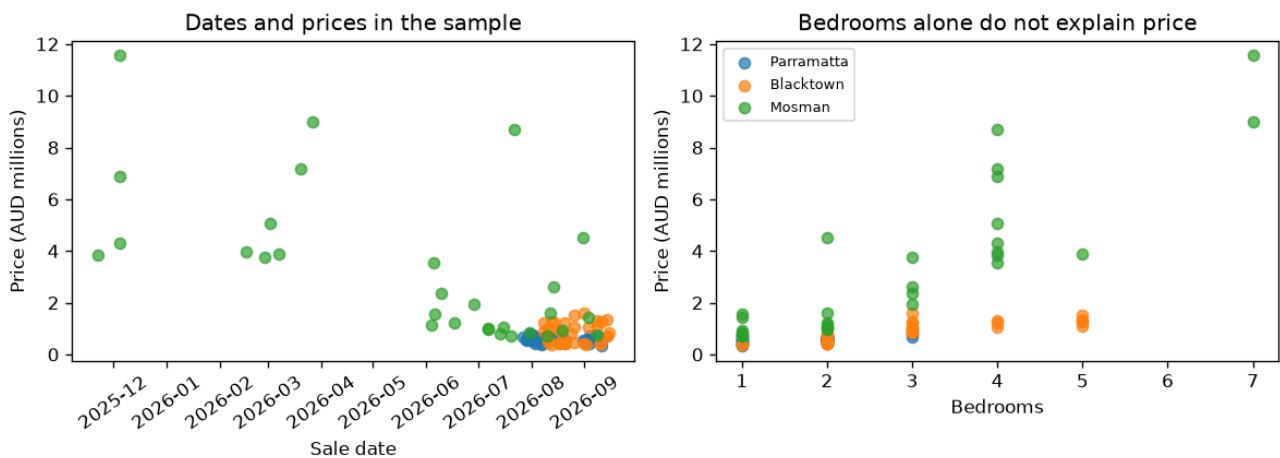


Figure 2. Sale dates and bedrooms. Older observations are mainly Mosman homes.

The date plot cannot establish a market trend because suburb and property mix change over time. Before engineering, the expected three strongest inputs were suburb, property type and bedrooms: market location, dwelling form and accommodation. Month index and bathrooms per bedroom were then added without using prices; studios use a denominator of one.

Within each cross-validation fold, missing numeric inputs use the training median, numeric features are standardised, and categories are one-hot encoded. This prevents preprocessing leakage. Scaling is essential for distance-based KNN. Address and listing identifiers are excluded.

3. Model development and evaluation

Three regression approaches were selected: KNN for similarity-based prediction, a decision tree for interpretable nonlinear rules, and random forest for variance reduction through averaging. Random forest was expected to perform best before training, although its predictions are less transparent than a single tree. Five shuffled folds evaluate a small parameter grid using MAE. RMSE emphasises large errors, while R-squared measures performance relative to a mean-price baseline.

Model	CV MAE	CV RMSE	CV R ²	Train MAE
KNN	\$437,074	\$888,508	0.625	\$317,782
Random forest	\$454,019	\$1,006,365	0.068	\$303,296
Decision tree	\$581,342	\$1,311,502	-5.053	\$376,494

Table 1. Mean fold scores at selected settings: KNN k=3; tree unrestricted depth/minimum leaf 3; forest 200 trees, unrestricted depth, minimum leaf 2 and max_features=0.8. KNN k=3/5/9, tree depth 2/4/unrestricted and forest depth 4/unrestricted were tested.

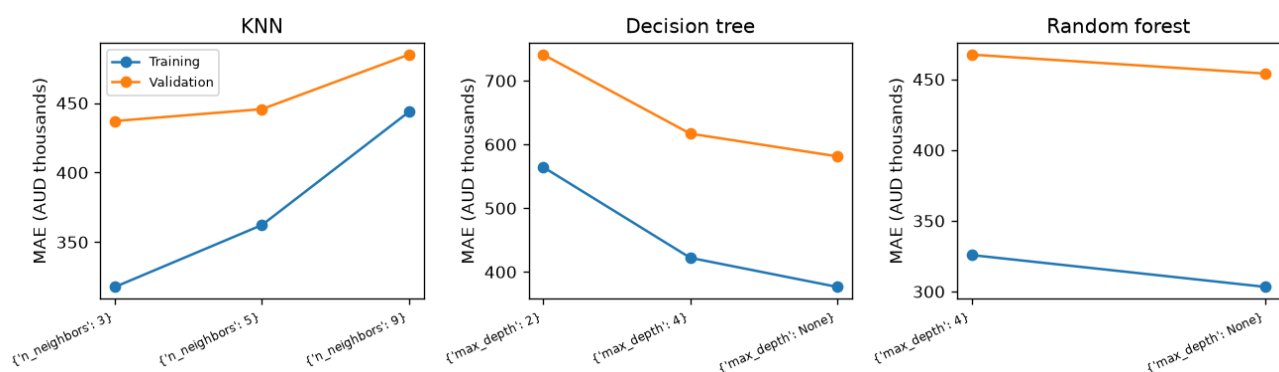


Figure 3. Training and validation MAE at each tested model complexity.

KNN achieved the lowest cross-validation MAE (\$437,074), contrary to the initial expectation. Its \$16,946 advantage over random forest is small relative to fold variability (SD \$207,485 and \$156,428, respectively). Increasing k raised training and validation errors, indicating excessive smoothing. Greater tree depth improved both scores within the tested range, although generalisation gaps remained. Fold-level R-squared was unstable because the small folds had different price variances.

24-property test	MAE	RMSE	R ²
KNN	\$1,202,750	\$3,731,553	0.384
Decision tree	\$1,208,407	\$3,509,637	0.455
Random forest	\$1,202,161	\$3,593,521	0.429
Median baseline	\$2,228,000	\$5,191,548	-0.193

KNN outperformed the median baseline on the test set, but its large absolute errors limit practical use. Random forest obtained slightly lower test MAE and the tree lower RMSE. The cross-validation selection was retained to avoid choosing a model using test outcomes. Because cross-validation also guided tuning, its reported scores may be optimistic. KNN was retained for deployment on this basis.

4. Investigating the five largest errors

Property	Actual	KNN estimate	Absolute error
17 Morella Road	\$23,000,000	\$5,391,000	\$17,609,000
10 Cyprian Street	\$8,500,000	\$5,391,000	\$3,109,000
53 - 53A Pelleas Street	\$1,700,000	\$4,800,000	\$3,100,000
96 Glover Street	\$3,390,000	\$5,383,333	\$1,993,333
13 Lancaster Street	\$1,900,000	\$1,278,333	\$621,667

17 Morella Road, Mosman: the \$23 million luxury sale exceeds the highest training price (\$11.55 million). Neighbour averaging cannot reach it. Harbour views, architectural design and bay access are missing. The retained sold-card bathroom count is three; a current profile says four.

10 Cyprian Street, Mosman: views, a pool and rebuilding potential are absent. Its encoded inputs match Morella exactly, including March 2026, so both receive \$5.391 million despite very different outcomes.

53-53A Pelleas Street, Blacktown: seven bedrooms and four bathrooms attract inappropriate neighbours: two Mosman houses at \$9 million and \$3.9 million, alongside a \$1.5 million local house. The model misses the local context of this dual-living home.

96 Glover Street, Mosman: the listing reports a 379 m² block, shared driveway and \$3,600 quarterly strata charges. These omitted details could help explain the overestimate. One neighbour costs \$8.7 million. This is a plausible explanation, not a proven causal effect.

13 Lancaster Street, Blacktown: a recent custom-built, multigenerational house is compared with three local sales between \$1.235 million and \$1.34 million. Construction age and finish quality are missing, so its premium is underestimated.

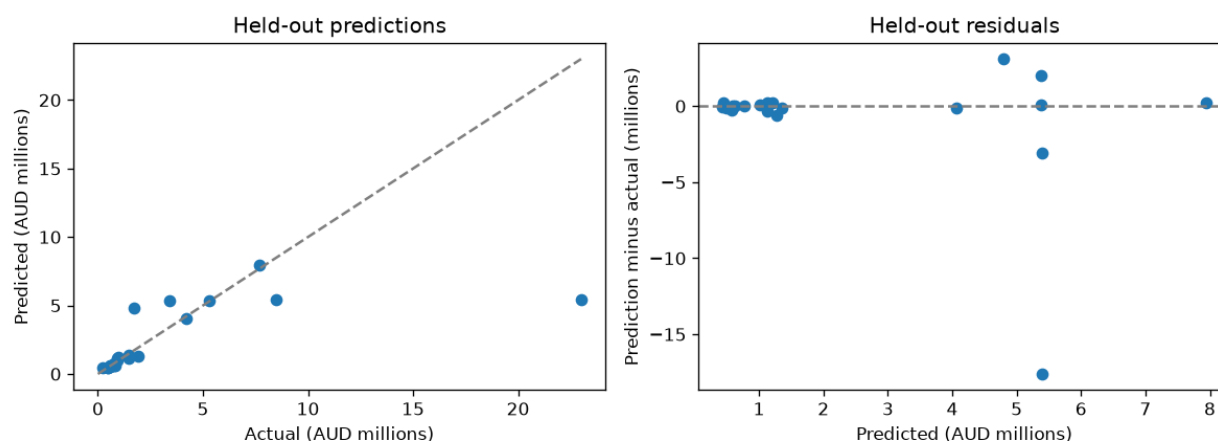


Figure 4. Held-out predictions and errors. Rare, expensive sales dominate squared error.

5. Comparison of valuation approaches

Ten held-out properties were assessed using the selected KNN model, a large language model (Codex) and personal estimates. The LLM estimates were recorded before the target prices were disclosed, although earlier market examples were available. Personal estimates were recorded after sale prices and reference estimates had been shown; their errors are therefore descriptive rather than evidence of independent forecasting accuracy.

ID / suburb	Actual	ML	LLM	Personal
C01 / Parramatta	\$250,000	\$443,000	\$310,000	\$345,000
C02 / Parramatta	\$685,000	\$572,333	\$810,000	\$785,000
C03 / Blacktown	\$465,000	\$438,000	\$450,000	\$495,000
C04 / Blacktown	\$900,000	\$1,131,000	\$1,150,000	\$1,180,000
C05 / Blacktown	\$1,700,000	\$4,800,000	\$1,750,000	\$1,425,000
C06 / Blacktown	\$590,000	\$499,667	\$520,000	\$580,000
C07 / Mosman	\$3,390,000	\$5,383,333	\$4,800,000	\$4,650,000
C08 / Mosman	\$5,300,000	\$5,383,333	\$4,700,000	\$5,250,000
C09 / Mosman	\$4,200,000	\$4,070,000	\$5,100,000	\$4,880,000
C10 / Mosman	\$23,000,000	\$5,391,000	\$4,800,000	\$5,350,000

Table 3. Sale prices and estimates in AUD. Property identifiers link to the full feature records in the accompanying dataset.

Approach	MAE	RMSE	R ²
ML	\$2,356,967	\$5,690,310	0.242
LLM	\$2,168,000	\$5,783,489	0.217
Personal estimate	\$2,043,000	\$5,601,331	0.265

Under these conditions, personal estimates produced the lowest numerical MAE (\$2,043,000) and RMSE (\$5,601,331). They were closest for four properties, compared with three each for the LLM and KNN. The personal estimates were particularly close for C06 and C08, while the LLM was closest for C05. All three approaches substantially underestimated the \$23 million sale.

The LLM achieved lower MAE than KNN, whereas KNN achieved lower RMSE. Human judgement may contribute inspection findings and local knowledge, but this comparison does not isolate those advantages. KNN is reproducible but constrained by its training data; LLM estimates may be plausible without adequate property-specific evidence. A stronger comparison would record all estimates before disclosing outcomes.

Revisiting the feature expectations

Validation permutation importance ranks suburb first, then month and bathrooms. The initial guess is partly supported: location leads, but type and bedrooms rank lower. Importance is not causal; correlated features share information, and month may identify the collection pattern rather than price growth.

6. Deployment and use

The fitted 95-row KNN pipeline is saved with joblib. A Flask endpoint uses that pipeline; a static HTML/JavaScript interface loads exported scaling, imputation, categories and neighbours. Python and browser predictions match for all 119 records. Equal-distance ties use a stable row order in both implementations. This consistency fix followed initial evaluation and did not tune test performance.

The screenshot shows the 'Sydney housing price predictor' web interface. The title bar reads 'SIT720 · MACHINE LEARNING MINI PROJECT'. Below the title is the subtitle 'Sydney housing price predictor' and a instruction 'Enter a property's details to estimate its sale price in Australian dollars.' The main form is divided into two sections: 'Property details' and 'Estimated sale price'. The 'Property details' section contains four input fields: 'Suburb' (a dropdown menu with 'Parramatta' selected), 'Property type' (a dropdown menu with 'Apartment' selected), 'Bedrooms' (a text input with '2'), and 'Bathrooms' (a text input with '2'). Below the 'Bedrooms' field is a small note 'Use 0 for a studio.' The 'Parking spaces' field is a text input with '1', and below it is a note 'Leave blank if unknown.' The 'Valuation date' field is a date picker showing '08/01/2026', with a note 'Dates in the data: Nov 2025 - Sep 2026.' A blue 'Predict price' button is at the bottom of the 'Property details' section. The 'Estimated sale price' section is currently empty, showing only the title and a sub-header 'Enter the details'.

Figure 5. Property input form before prediction.

This screenshot shows the same web interface as Figure 5, but now the 'Estimated sale price' section displays the result. The estimated price is '\$692,333' in large blue text. Below the price, the input details are summarized: 'Parramatta · Apartment · 2 bedrooms'. The 'About this estimate' section remains the same, stating that the model averages the prices of 3 similar training examples and that it was trained on 95 properties and tested on 24, with an average absolute test error of about \$1.20 million.

Figure 6. Prediction for a Parramatta apartment: 2 bedrooms, 2 bathrooms, 1 parking space, 1 August 2026.

Choose suburb and type, enter room counts, parking and date, then click Predict price. Zero bedrooms means studio; blank parking means unknown. The app displays the estimate, model description and test error. Dates beyond training coverage trigger a warning. The app is publicly hosted on GitHub Pages and requires no sign-in.

Reproduction and reflection

Build and run

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python train.py
python export_model.py
python verify_project.py
python build_report.py
python app.py
```

Use the extracted project folder. On Windows activate with `.venv\Scripts\activate`. Open `http://127.0.0.1:5000`. For notebook outputs, select this environment in Jupyter/VS Code and run all cells. The static folder `site/dist` can also be hosted directly. Verification needs Node.js. Full setup and archive instructions are in `README.md`.

Reflection

The main finding is that data coverage and feature quality constrain performance more than model complexity alone. Random forest did not achieve the expected advantage, and KNN's favourable cross-validation result did not prevent large test errors. Missing information about area, views, condition and ownership costs limits the relationships available to each model. Further work should prioritise consistent property details, building-grouped splits and forward-time evaluation.

Deployment required identical preprocessing and distance tie handling in Python and JavaScript. A simple interface makes the prediction process accessible, but additional computing alone cannot compensate for missing information. Suburb may reflect historic inequality, while selective disclosure may underrepresent particular housing types. Test MAE was \$100,278 for Parramatta, \$504,389 for Blacktown and \$2,636,093 for Mosman. With only six to nine cases per group, these results cannot establish fairness or justify lending decisions.

Appendix: core code

Core steps adapted from train.py. The executed notebook and all source files in the archive contain the complete workflow, outputs, plots and application. See stable_knn.py for deterministic tie handling.

```
def make_features(frame):
    features = frame[["suburb", "property_type", "bedrooms",
                     "bathrooms", "parking"]].copy()
    dates = pd.to_datetime(frame["sale_date"])
    features["sale_month"] = (dates.dt.year - 2025) * 12 + dates.dt.month - 1
    features["bathrooms_per_bedroom"] = (
        features["bathrooms"] / features["bedrooms"].clip(lower=1))
    return features

numeric = ["bedrooms", "bathrooms", "parking", "sale_month",
           "bathrooms_per_bedroom"]
categorical = ["suburb", "property_type"]
preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     categorical)
])
cv = KFold(n_splits=5, shuffle=True, random_state=42)

# Example: KNN selection. The notebook repeats this for all models.
pipe = Pipeline([("preprocess", preprocess),
                 ("model", StableKNNRegressor())])
search = GridSearchCV(pipe, {"model__n_neighbors": [3, 5, 9]},
                      scoring={"mae": "neg_mean_absolute_error",
                              "rmse": "neg_root_mean_squared_error", "r2": "r2"},
                      refit="mae", cv=cv, return_train_score=True)
search.fit(X_train, y_train)

predictions = best_model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
joblib.dump(best_model, ROOT / "model/housing_model.joblib")
```

The appendix abbreviates variable setup and the model loop; run the notebook or train.py for a complete runnable program. The saved pipeline includes preprocessing, preventing separate application transformations from drifting away from training.

Application structure

File	Responsibility
app.py	Serve the page and POST /api/predict
prediction.py	Validate input, engineer features and call Python pipeline
export_model.py	Export fitted KNN preprocessing and neighbour data
site/dist/predict.js	Apply the same transformations and stable KNN average
site/dist/app.js	Read the form, validate and display result
verify_project.py	Check data, held-out split, API and Python/JavaScript parity